Reporte Técnico de Actividades Práctico-Experimentales Nro 4.

1. Datos de Identificación del Estudiante y la Práctica

Nombre del estudiante(s)	Ivett Zaragocin; Anthony Yaguana; Pedro Jiménez; Steven Pardo
Asignatura	Simulación
Ciclo	5
Unidad	1
Resultado de aprendizaje de la unidad	**R1.** Aplica análisis estadísticos de los datos de la Simulación para mejorar la calidad de los resultados, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	4
Título de la Práctica	Análisis comparativo de herramientas de simulación mediante un modelo común
Nombre del Docente	José O. Guamán Q.
Fecha	22 de Octubre de 2026 23 de Octubre 2026
Horario	Jueves 10h30 a 11h30 Viernes 07h30 a 09h30
Lugar	Aula
Tiempo planificado en el Sílabo	3

2. Objetivo(s) de la Práctica

Comparar diferentes lenguajes y plataformas para la implementación de modelos de simulación, a partir de un modelo epidemiológico SIR común.

- Implementar un modelo SIR sencillo en Python con SimPy y en NetLogo.
- Comparar la sintaxis, la facilidad de implementación y la visualización de cada herramienta.
- Identificar el ámbito de aplicación de cada herramienta.

3. Materiales, Reactivos, Equipos y Herramientas

- Computador.
- Python 3.12.
- Biblioteca SimPy 4.1.

- Matplotlib 3.11.
- NetLogo 7.0 con la extensión csv.
- Git y GitHub.
- Editor de código: Visual Studio Code.
- Sistema operativo Linux.

4. Procedimiento / Metodología Ejecutada

Fundamento teórico

El modelo SIR, propuesto por Kermack y McKendrick [1], es un modelo compartimental que describe la propagación de una enfermedad infecciosa en una población cerrada. Cada individuo pertenece a uno de tres compartimentos: susceptibles (S), que todavía pueden contagiarse; infectados (I), que tienen la enfermedad y pueden transmitirla; y recuperados (R), que ya superaron la infección y no vuelven a contagiarse [2]. Como la población no cambia de tamaño durante la simulación, en todo instante se cumple:

$$N = S + I + R \quad (1)$$

La evolución del sistema depende de dos parámetros: la tasa de transmisión β, que mide la probabilidad de contagio en un contacto entre un infectado y un susceptible, y la tasa de recuperación γ, que mide la fracción de infectados que se recupera por unidad de tiempo [2]. En su forma continua, el modelo se expresa con el siguiente sistema de ecuaciones diferenciales:

$$dS/dt = -\beta \cdot S \cdot I / N \quad (2) \text{ variación de susceptibles}$$

$$dI/dt = \beta \cdot S \cdot I / N - \gamma \cdot I \quad (3) \text{ variación de infectados}$$

$$dR/dt = \gamma \cdot I \quad (4) \text{ variación de recuperados}$$

El término $\beta \cdot S \cdot I/N$ representa los nuevos contagios por unidad de tiempo y $\gamma \cdot I$ los infectados que se recuperan. Así, el flujo de individuos es siempre $S \rightarrow I \rightarrow R$: un susceptible se infecta ($S + I \rightarrow I + I$) y un infectado se recupera ($I \rightarrow R$).

El número reproductivo básico indica cuántas infecciones produce, en promedio, un infectado al inicio de la epidemia, cuando casi toda la población es susceptible [2]:

$$R_0 = \beta / \gamma = 0.30 / 0.10 = 3 \quad (5)$$

Como $R_0 > 1$, se espera que la infección se propague: el número de infectados crece, alcanza un máximo y luego disminuye a medida que se agotan los susceptibles [2].

En lugar de resolver las ecuaciones de forma analítica, la práctica usa una simulación discreta basada en individuos: el tiempo avanza en pasos de una unidad y en cada paso se decide al azar quién se contagia y quién se recupera [3]. Por eso, los resultados dependen del generador de números aleatorios. Para que sean reproducibles se fija una semilla, que hace que el generador produzca siempre la misma secuencia [3].

Se utilizaron dos herramientas. SimPy es una biblioteca de Python para simulación de eventos discretos basada en procesos: cada proceso es una función generadora que avanza el reloj de la simulación con env.timeout() [4]. NetLogo es un entorno de modelado basado en agentes, en el que cada individuo es un agente (turtle) con su propio estado y la simulación avanza por ticks [5]. Python

genera los datos y los exporta en un archivo CSV, que NetLogo importa para visualizarlos y compararlos con su propio modelo.

La tabla 1 resume los parámetros utilizados, que son los mismos en ambas herramientas.

Parámetro	Símbolo	Valor
Población total	N	100
Infectados iniciales	I_0	5
Susceptibles iniciales	S_0	95
Tasa de infección	β	0.30
Tasa de recuperación	γ	0.10
Tiempo de simulación	t	0 a 100
Semilla aleatoria	—	42
Número reproductivo básico	R_0	3

Tabla 1. Parámetros del modelo SIR

Metodología ejecutada

La práctica se desarrolló en Python 3.12 con SimPy y Matplotlib, y en NetLogo 7.0, siguiendo estas etapas:

- **Análisis de la guía.** Se revisaron el modelo SIR, sus parámetros, el pseudocódigo común y el formato del archivo CSV que NetLogo debe importar. Por acuerdo del grupo, la comparación se realizó entre Python/SimPy y NetLogo, sin la implementación en Java.
- **Acuerdo de reglas comunes.** Antes de programar se fijaron los datos que todos usan (tabla 1), la regla de cada paso, en la que cada infectado elige un contacto al azar, y la cabecera del CSV con las columnas tiempo, susceptibles, infectados y recuperados.
- **Creación del repositorio.** Se creó el repositorio en GitHub (github.com/ivecanahi/ape4-grupal) con la carpeta python/ y el archivo requirements.txt.
- **División del trabajo.** La parte en Python se dividió en cuatro partes (tabla 2): datos y población, simulación con SimPy, salida de resultados, e integración y pruebas. La implementación en NetLogo se realizó de forma individual.
- **Desarrollo con pruebas.** Se escribieron 13 pruebas automatizadas con unittest que verifican el modelo, el archivo CSV, la gráfica y el reporte.
- **Integración y corrección.** Al integrar las partes se detectó que el controlador importaba un módulo que había cambiado de nombre, por lo que el programa no se ejecutaba. Se unieron la Parte 1 y la Parte 2 en el archivo model.py, como indica la guía, y se unificó la forma de importar los módulos.
- **Reporte visual.** Se añadió una vista que genera un reporte HTML con la explicación de la práctica, ilustraciones del modelo, las métricas, la gráfica y la tabla de datos.

- **Integración con NetLogo.** Se copió resultados_python.csv a la carpeta del modelo de NetLogo y se importó con el botón CARGAR PYTHON para compararlo con el modelo propio.
- **Ejecución y análisis.** Se ejecutó la simulación, se analizaron las curvas S(t), I(t) y R(t) y se compararon las dos herramientas.

Parte	Archivo	Contenido
1. Datos y población	python/model.py (parte superior)	State, SIRParameters, StepResult y Population.
2. Simulación con SimPy	python/model.py (parte inferior)	SIRSimulation: registro, contagio, recuperación y proceso de SimPy.
3. Salida	python/view.py	CsvExporter, ChartView y ReportView.
4. Integración y pruebas	controller.py, main.py y test_model.py	SIRController, programa principal y pruebas.

Tabla 2. División del trabajo en Python

El proyecto sigue el patrón Modelo-Vista-Controlador (MVC): el modelo calcula la epidemia, las vistas presentan los resultados y el controlador conecta ambos. Así, la generación de datos queda separada de su visualización. La tabla 3 describe los módulos.

Módulo	Función
python/model.py	Modelo: estados, parámetros, población y simulación SIR con SimPy.
python/view.py	Vistas: exporta el CSV y genera la gráfica S(t), I(t), R(t) y el reporte HTML.
python/controller.py	Controlador: ejecuta la simulación y entrega los resultados a cada vista.
python/main.py	Programa principal: ejecuta todo y abre el reporte en el navegador.
python/test_model.py	Pruebas del modelo y del controlador.
python/test_view.py	Pruebas del CSV, la gráfica y el reporte.

Tabla 3. Módulos del proyecto en Python

La simulación en Python sigue el pseudocódigo de la guía y funciona así:

- Se crea un generador de números aleatorios random.Random(42) y una población de 100 individuos en estado S; con rng.sample se eligen 5 al azar y se pasan a I.
- Se crea el entorno de SimPy (simpy.Environment) y un proceso que se repite desde t = 0 hasta t = 100.
- En cada paso se registran el tiempo y los conteos de S, I y R, y se toma una copia del estado de la población.
- Cada individuo que estaba infectado en la copia elige un contacto al azar, distinto de sí mismo. Si el contacto era susceptible y un número aleatorio es menor que $\beta = 0.30$, se contagia.
- Después, cada individuo que estaba infectado en la copia se recupera si un número aleatorio es menor que $\gamma = 0.10$.

- El proceso ejecuta yield env.timeout(1) para avanzar el reloj una unidad. Al llegar a t = 100 se registra la última fila, con lo que se obtienen 101 registros.
- Al terminar, el controlador guarda resultados_python.csv, la gráfica grafica_sir_python.png y el reporte reporte_python.html.

Los cambios se calculan sobre la copia del estado para que un individuo contagiado en un paso no pueda contagiar a otro en ese mismo paso.

En NetLogo se creó el modelo modelo_sir_netlogo.nlogox (formato de NetLogo 7) con 100 agentes (turtles) y los mismos parámetros. Su funcionamiento es el siguiente:

- El procedimiento setup fija random-seed 42, crea 100 agentes en estado S en posiciones al azar y, con ask n-of 5, pasa 5 de ellos a I.
- En cada tick, go guarda primero los conjuntos de agentes que eran I y S al inicio del tick, que cumplen el papel de la copia del estado. Cada infectado elige un contacto con one-of other turtles; si el contacto era S y random-float 1 es menor que β, se contagia. Después, cada infectado se recupera si random-float 1 es menor que γ.
- Los agentes se mueven ligeramente en cada tick solo con fines visuales, ya que el contacto se elige al azar en toda la población, igual que en Python. Su color indica el estado: verde (S), rojo (I) y gris (R).
- El botón CARGAR PYTHON usa la extensión csv: csv:from-file lee resultados_python.csv, but-first elimina la cabecera y plotxy dibuja cada fila en el gráfico Python y en tres gráficos de comparación (susceptibles, infectados y recuperados), junto a las curvas del modelo de NetLogo.

Para reproducir los resultados se ejecuta pip install -r requirements.txt y luego python main.py dentro de la carpeta python/. El programa imprime el estado final y genera los tres archivos de salida. Las pruebas se ejecutan con python -m unittest.

5. Resultados

Resultados en Python

La simulación produjo 101 registros, de t = 0 a t = 100, y se ejecutó en menos de un segundo, incluida la generación de los archivos. La tabla 4 muestra el estado de la población cada 10 unidades de tiempo; en todos los instantes se cumple S + I + R = 100.

t	S	I	R	S + I + R
0	95	5	0	100
10	64	21	15	100
20	27	30	43	100
30	10	21	69	100
40	4	14	82	100
50	3	10	87	100
60	2	3	95	100
70	2	0	98	100
80	2	0	98	100
90	2	0	98	100
100	2	0	98	100

Tabla 4. Evolución de la población cada 10 unidades de tiempo (Python, semilla 42)

Métrica	Valor	Interpretación
Pico de infectados	34 en t = 16	Máximo de personas enfermas al mismo tiempo.
Fin del brote	t = 70	Primer instante sin infectados.
Tasa de ataque	98 %	Porcentaje de la población que se infectó alguna vez.
Susceptibles finales	2	Personas que nunca se contagiaron.
Recuperados finales	98	Personas que superaron la infección.

Tabla 5. Métricas de la simulación en Python

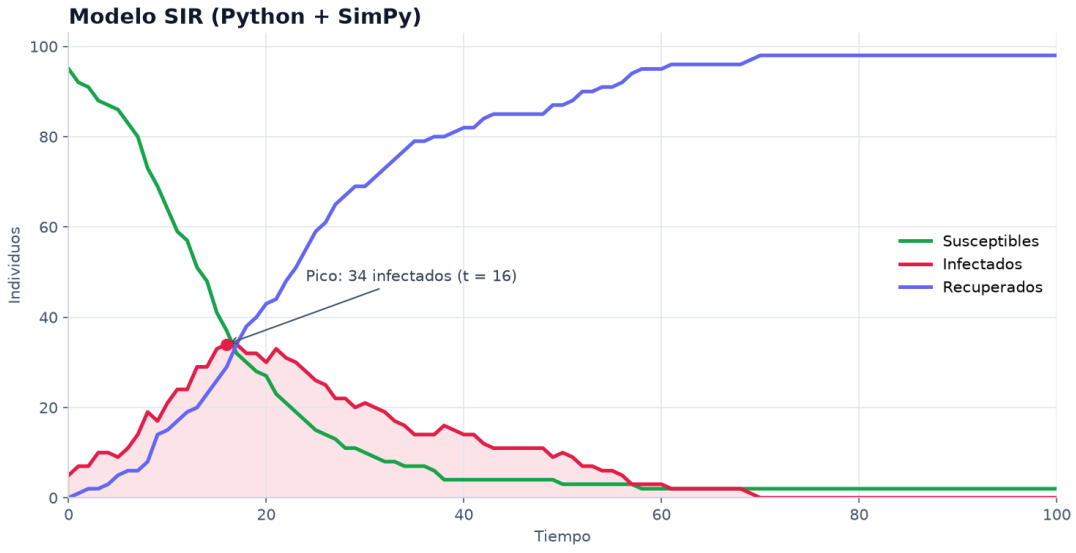


Figura 1. Evolución de S(t), I(t) y R(t) en Python con SimPy

La figura 1 muestra el comportamiento esperado para $R_0 = 3$. Los susceptibles disminuyen rápidamente durante las primeras 20 unidades de tiempo, los infectados crecen hasta un máximo de 34 en t = 16 y luego disminuyen, y los recuperados aumentan de forma continua hasta estabilizarse en 98. Las curvas de

susceptibles y recuperados se cruzan cerca del pico de infectados: desde ese momento cada infectado encuentra cada vez menos susceptibles, por lo que los nuevos contagios ya no compensan las recuperaciones. A partir de t = 70 no quedan infectados y el sistema ya no cambia.

Que 2 personas nunca se contagien también es esperable: en el modelo SIR la epidemia termina por falta de infectados, no por falta de susceptibles [2].

Reporte visual

Además del CSV y la gráfica, el programa genera un reporte HTML que explica la práctica con ilustraciones. Las figuras 2 a 4 muestran algunas de sus secciones.



Figura 2. Estados del modelo SIR en el reporte generado por Python



Figura 3. Reglas de contagio y recuperación de cada paso

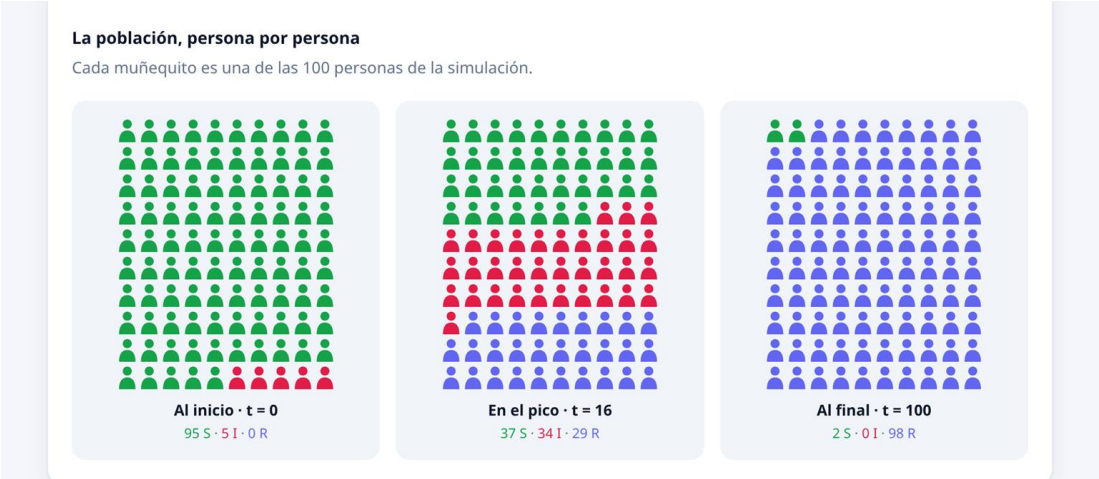


Figura 4. Población al inicio, en el pico y al final de la simulación

Resultados en NetLogo

El modelo de NetLogo se ejecutó con los mismos parámetros y la semilla 42 durante 100 ticks. La tabla 6 muestra la evolución de la población cada 10 ticks; también se cumple S + I + R = 100 en todo instante.

t	S	I	R	S + I + R
0	95	5	0	100
10	59	30	11	100
20	27	29	44	100
30	16	11	73	100
40	11	6	83	100
50	8	3	89	100
60	5	3	92	100
70	5	1	94	100
80	5	1	94	100
90	5	0	95	100
100	5	0	95	100

Tabla 6. Evolución de la población cada 10 ticks (NetLogo, semilla 42)

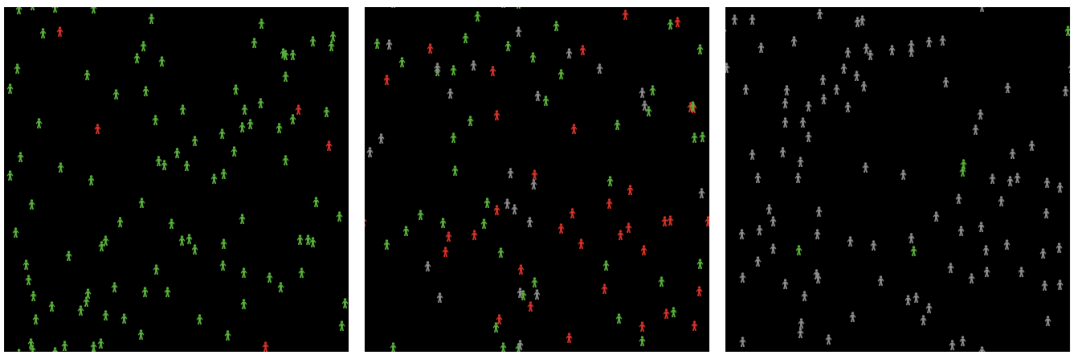


Figura 5. Vista del modelo en NetLogo al inicio, en el pico y al final (verde: S, rojo: I, gris: R)

La figura 5 muestra a los agentes en tres momentos. Al inicio hay 5 infectados entre 95 susceptibles; en el pico (t = 14) conviven los tres estados, con 35 infectados; y al final casi toda la población está recuperada y solo 5 agentes nunca se contagiaron.

Después de ejecutar el modelo se presionó CARGAR PYTHON para importar resultados_python.csv. La figura 6 compara las dos simulaciones para cada estado.

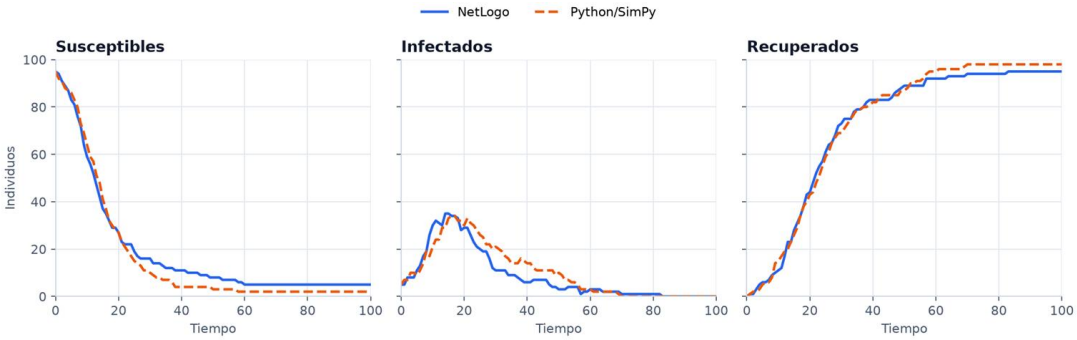


Figura 6. Comparación de susceptibles, infectados y recuperados entre NetLogo y Python/SimPy

Métrica	NetLogo	Python/SimPy
Pico de infectados	35 en t = 14	34 en t = 16
Fin del brote	t = 83	t = 70
Tasa de ataque	95 %	98 %
Susceptibles finales	5	2
Recuperados finales	95	98

Tabla 7. Métricas de NetLogo y Python/SimPy con la misma semilla

Las dos curvas tienen la misma forma: los susceptibles caen rápidamente durante las primeras 20 unidades de tiempo, los infectados forman un único pico de tamaño casi igual (35 frente a 34) y los recuperados crecen hasta estabilizarse por encima de 95. Las diferencias aparecen en los detalles: el pico de NetLogo ocurre dos ticks antes, su brote se extiende hasta t = 83 por un infectado que tarda en recuperarse, y al final quedan 5 susceptibles en lugar de 2. Estas diferencias no indican un error en ninguna de las dos implementaciones, sino que provienen de la generación de números aleatorios, como se explica más adelante.

Comparación de herramientas

La tabla 8 compara ambas herramientas a partir de la experiencia obtenida en la práctica.

Característica	NetLogo	Python/SimPy
Facilidad de implementación	El modelo completo, con la importación del CSV, ocupa menos de 80 líneas: los agentes, el ciclo setup/go y los gráficos ya están integrados.	Requiere programar la población, el ciclo de la simulación y la exportación; se organizó en clases y módulos.
Manejo de agentes	Nativo: cada individuo es una tortuga con su propio estado, posición y color.	No es nativo: la población se representó como una lista de estados.
Generación de datos	Genera los datos de cada tick, pero exportarlos requiere comandos adicionales, como export-plot o BehaviorSpace.	Genera los datos y los guarda en CSV con pocas líneas de código.
Visualización	Muestra los agentes en tiempo real y tiene gráficos y monitores integrados en la interfaz.	Gráficas configurables con Matplotlib y reporte HTML; no muestra la simulación en tiempo real.
Flexibilidad	Limitada al lenguaje de NetLogo y a sus extensiones (por ejemplo, csv).	Alta: se combina con cualquier biblioteca de Python y se puede probar con unittest.
Tiempo de ejecución	Los 100 ticks tardan unos 0.4 s en modo headless; en la interfaz depende de la velocidad elegida.	La simulación tarda unos 3 ms y, con la gráfica y el reporte, menos de 1 s.
Integración con otras herramientas	Importa archivos CSV con la extensión csv incluida en NetLogo.	Exporta CSV, PNG y HTML que otras herramientas pueden leer.

Tabla 8. Comparación entre NetLogo y Python/SimPy

Diferencias por la generación de números aleatorios

Aunque ambas herramientas usan el mismo modelo, los mismos parámetros y la misma semilla (42), sus resultados no coinciden exactamente: el pico fue de 35 infectados en t = 14 en NetLogo y de 34 en t = 16 en Python. Ambas usan generadores basados en el algoritmo Mersenne Twister [6], [7], pero cada una inicializa el generador a partir de la semilla a su manera y consume los números aleatorios en un orden distinto. En NetLogo, la instrucción ask recorre los agentes en un orden aleatorio en cada tick, lo que también consume números del generador; en Python, la lista de individuos se recorre siempre en el mismo orden. Además, cada implementación hace las llamadas al azar (elegir un contacto, decidir un contagio o una recuperación) en una secuencia diferente.

Por eso las dos simulaciones son realizaciones distintas del mismo proceso aleatorio y no deben compararse valor por valor. Lo que sí debe coincidir es el comportamiento general: la forma de las curvas, la existencia de un pico de infectados y que casi toda la población termine recuperada.

6. Preguntas de Control

1. ¿Qué representa el modelo SIR?

Representa la propagación de una enfermedad infecciosa en una población cerrada, dividida en susceptibles, infectados y recuperados, y cómo cambia el tamaño de cada grupo con el tiempo.

2. ¿Qué representan los estados S, I y R?

S agrupa a las personas sanas que pueden contagiarse; I, a las personas enfermas que pueden contagiar; y R, a las personas que se recuperaron y ya no participan en el contagio.

3. ¿Qué función cumplen los parámetros β y γ?

β controla la velocidad de los contagios: en el código es la probabilidad de que el contacto entre un infectado y un susceptible produzca un contagio. γ controla la velocidad de recuperación: es la probabilidad de que un infectado se recupere en cada paso. Su cociente β/γ es el número reproductivo básico R_0.

4. ¿Por qué es importante utilizar los mismos parámetros en las tres herramientas?

Porque la comparación solo es válida si la única diferencia es la herramienta. Si los parámetros cambian, no se puede saber si una diferencia en los resultados se debe al modelo o a la herramienta.

5. ¿Por qué Python, Java y NetLogo pueden producir resultados diferentes utilizando el mismo modelo?

Porque la simulación es estocástica y cada herramienta inicializa y consume los números aleatorios de forma distinta, como se explicó en la sección de resultados. Cada ejecución es una realización diferente del mismo proceso: coinciden en el comportamiento general, no en los valores exactos.

6. ¿Qué ventajas ofrece NetLogo para representar sistemas basados en agentes?

Cada agente tiene su propio estado y posición, la interfaz muestra la simulación en tiempo real y el lenguaje incluye instrucciones como ask, one-of o with, que facilitan escribir reglas para cada agente.

7. ¿Qué ventajas ofrece SimPy para desarrollar simulaciones basadas en eventos?

Gestiona el reloj de la simulación y la ejecución de los procesos: el modelador solo describe qué hace cada proceso y cuándo espera (env.timeout). Además, permite usar todo el ecosistema de Python para pruebas, gráficas y exportación de datos.

8. ¿Qué ventajas ofrece Java para implementar modelos de simulación?

Aunque en esta práctica no se implementó, Java ofrece alto rendimiento por ser compilado, tipado estático que detecta errores antes de ejecutar y buenas herramientas para organizar proyectos grandes.

9. ¿Qué función cumple el archivo CSV en la integración Python/Java → NetLogo?

Es el formato intermedio que permite que NetLogo lea los datos generados en Python. Es un archivo de texto simple que ambas herramientas entienden, con una cabecera común acordada: tiempo,susceptibles,infectados,recuperados.

10. ¿Qué ventajas presenta separar la generación de datos de la visualización?

Cada herramienta hace lo que mejor sabe: Python genera los datos y NetLogo los visualiza. Además, se puede cambiar la visualización sin tocar el modelo, comparar varias fuentes de datos en una misma vista y verificar el modelo con

pruebas automáticas. En el proyecto, esta separación también se aplicó con el patrón MVC.

7. Conclusiones

- Se implementó el modelo SIR en Python con SimPy siguiendo el pseudocódigo de la guía. Con $\beta = 0.30$, $\gamma = 0.10$ y semilla 42 se obtuvo un pico de 34 infectados en $t = 16$ y una tasa de ataque del 98 %, resultados coherentes con $R_0 = 3$.
- La simulación discreta reproduce el comportamiento del modelo de ecuaciones diferenciales: los susceptibles disminuyen, los infectados alcanzan un pico y los recuperados crecen hasta estabilizarse.
- El modelo equivalente en NetLogo produjo un pico de 35 infectados en $t = 14$ y una tasa de ataque del 95 %. Ambas herramientas reproducen el mismo comportamiento, con diferencias pequeñas en los valores exactos.
- Python con SimPy resulta más adecuado para generar y procesar datos, automatizar pruebas y exportar resultados, mientras que NetLogo resulta más adecuado para representar agentes y visualizar la simulación de forma interactiva.
- El archivo CSV con una cabecera común permitió integrar ambas herramientas y separar la generación de datos de su visualización.
- Dos implementaciones del mismo modelo estocástico no producen los mismos valores aunque usen la misma semilla, porque inicializan y usan sus generadores de números aleatorios de forma distinta; por eso la comparación debe hacerse sobre el comportamiento general.
- Las pruebas automatizadas permitieron detectar y corregir el error de integración entre los módulos del proyecto.

Referencias

[1] W. O. Kermack y A. G. McKendrick, “A contribution to the mathematical theory of epidemics,” *Proceedings of the Royal Society of London. Series A*, vol. 115, no. 772, pp. 700–721, 1927, doi: 10.1098/rspa.1927.0118.

[2] H. W. Hethcote, “The mathematics of infectious diseases,” *SIAM Review*, vol. 42, no. 4, pp. 599–653, 2000, doi: 10.1137/S0036144500371907.

[3] J. Banks, J. S. Carson, B. L. Nelson y D. M. Nicol, *Discrete-Event System Simulation*, 5th ed. Upper Saddle River, NJ, USA: Pearson, 2010.

[4] Team SimPy, “SimPy: Discrete event simulation for Python,” *SimPy Documentation*. [En línea]. Disponible en: https://simpy.readthedocs.io/

[5] U. Wilensky, “NetLogo,” Center for Connected Learning and Computer-Based Modeling, Northwestern University, Evanston, IL, USA, 1999. [En línea]. Disponible en: https://ccl.northwestern.edu/netlogo/

[6] M. Matsumoto y T. Nishimura, “Mersenne Twister: A 623-dimensionally equidistributed uniform pseudo-random number generator,” *ACM Transactions on Modeling and Computer Simulation*, vol. 8, no. 1, pp. 3–30, 1998, doi: 10.1145/272991.272995.

[7] Python Software Foundation, "random — Generate pseudo-random numbers," *Python 3.12 Documentation*. [En línea]. Disponible en: https://docs.python.org/3.12/library/random.html

8. Declaración de uso de Inteligencia Artificial

En la elaboración de esta práctica se utilizó la herramienta de inteligencia artificial Claude, modelo Claude Opus 5.5, desarrollado por Anthropic, como apoyo en la integración y documentación del código en Python, el reporte HTML con ilustraciones. El resto del desarrollo fue realizado por los integrantes. Todo el contenido fue revisado, verificado y ajustado por la autora, quien asume la responsabilidad sobre la versión final.

Herramienta: Claude Opus 5.5 (Anthropic).

Uso: integración y documentación del código, reporte visual, modelo de NetLogo y apoyo en la redacción técnica del informe.

Prompts utilizados:

1. **Reporte visual:** *«Implementa una vista que permita visualizar de forma clara y explicativa como funciona la práctica.»*

9. Anexos

Link en Github:

https://github.com/ivecanahi/simulacion-practicas.git

Link de Video:

https://drive.google.com/file/d/1M0P-25usoKaka5Wd2qe_ntZqqmxCcoeG/view?usp=sharing